

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital | Carrera de Ingeniería de Software

Material de Clase — Semana 5
Contenedores y Orquestación de Servicios**Datos de la semana**

Asignatura	Aplicaciones Distribuidas (ISR-701) — Séptimo semestre
Unidad	2: Arquitecturas Distribuidas
Tema	3: Contenedores y Orquestación de Servicios
Semana	5 de 18 Fechas: 01 – 07 de junio de 2026
Subtemas	(1) Virtualización y contenerización: Docker; (2) Composición de servicios: Docker Compose; (3) Introducción a Kubernetes: pods, deployments y services; (4) Patrones de despliegue distribuido (blue-green, canary)
Docente	Prof. Guerrero Ulloa G.C.

Resultado de aprendizaje de la unidad

Aplica arquitecturas y patrones de diseño distribuido —microservicios, API REST, mensajería asíncrona, arquitectura en capas— para construir sistemas **escalables, disponibles y seguros**. Esta semana operacionaliza ese resultado: empaquetar el microservicio en un *artefacto inmutable* (imagen), componer el sistema completo de forma reproducible y orquestarlo con tolerancia a fallos y despliegues sin interrupción.

Nota de coherencia curricular

El temario de esta semana corresponde, en el libro guía (Guerrero Ulloa, 2026), al **Capítulo 7: “Contenedores Docker y Orquestación”**. La numeración de capítulos del libro avanza tema a tema; el *cronograma* de la asignatura agrupa Docker, Compose y Kubernetes como Tema 3 de la Unidad 2 (Arquitecturas Distribuidas), que es el orden seguido aquí. El microservicio de ejemplo, `svc-estudiantes`, es el mismo construido en la semana previa (Spring Boot + Java), de modo que esta clase lo *empaqueta y despliega* sin re-escribir lógica de negocio.

1. Mapa de la semana y objetivos operativos

Las tres semanas de la Unidad 2 trazan una progresión natural: primero se decide *cómo se estructura* el sistema (monolito a microservicios), luego *cómo se comunican* sus partes (APIs REST) y, finalmente —esta semana—, *cómo se empaqueta, se compone y se despliega* ese sistema en máquinas reales con garantías de disponibilidad. El hilo conductor es la palabra **reproducibilidad**: el mismo artefacto debe ejecutarse idéntico en la laptop del estudiante, en el laboratorio y en un servidor de producción.

Al finalizar la semana, el estudiante será capaz de: (i) explicar por qué los contenedores resuelven el problema “en mi máquina funciona” mejor que las máquinas virtuales; (ii) escribir un `Dockerfile` multi-etapa correcto y construir una imagen del microservicio desde IntelliJ IDEA; (iii) componer el sistema (microservicio + base de datos + gateway) con un único archivo `compose.yaml`; (iv) desplegar ese sistema en un clúster Kubernetes describiendo `Deployments` y `Services`; y (v) razonar sobre los patrones *blue-green* y *canary* para liberar versiones nuevas sin tiempo de caída.

Glosario de acrónimos usados en la semana

VM Virtual Machine (máquina virtual) • **SO** Sistema Operativo • **CLI** Command-Line Interface • **JDK** Java Development Kit • **JRE** Java Runtime Environment • **JAR** Java ARchive • **API** Application Programming Interface • **REST** Representational State Transfer • **YAML** YAML Ain't Markup Language • **IDE** Integrated Development Environment • **K8s** Kubernetes • **Pod** unidad mínima de despliegue en K8s • **etcd** almacén clave-valor del plano de control • **PVC** PersistentVolumeClaim • **CRD** Custom Resource Definition • **SLA** Service Level Agreement.

2. Subtema 1 — Virtualización y contenerización: Docker

2.1. El problema que resuelve un contenedor

Todo desarrollador ha vivido el síndrome “en mi máquina funciona”. Ocurre porque una aplicación no depende solo de su código, sino de un *entorno*: versión exacta del runtime (por ejemplo Java 25), variables de entorno, librerías del sistema, configuración de red y rutas de archivos. Cuando ese entorno difiere entre la laptop, el laboratorio y el servidor, el comportamiento diverge. La **contenerización** ataca el problema de raíz: empaqueta la aplicación *junto con todas sus dependencias* en una unidad estándar y portátil llamada **imagen**, que se ejecuta de forma idéntica en cualquier host con un motor de contenedores instalado.

Definición: contenedor

Un **contenedor** es un proceso (o conjunto de procesos) aislado del resto del sistema mediante primitivas del núcleo de Linux —*namespaces* (aislamiento de visión: procesos, red, montajes) y *cgroups* (control de recursos: CPU, memoria, E/S)—, que comparte el **núcleo del sistema operativo anfitrión** en lugar de virtualizar hardware completo. Esto lo hace ligero (megabytes, no gigabytes), de arranque casi instantáneo y reproducible [1, 2].

2.2. Máquinas virtuales frente a contenedores

La diferencia esencial es *qué* se virtualiza. Una máquina virtual virtualiza **hardware** mediante un hipervisor y ejecuta un sistema operativo invitado completo por cada instancia; un contenedor virtualiza el **sistema operativo** y reutiliza el núcleo del anfitrión. La consecuencia práctica es enorme: donde un servidor aloja una decena de VMs, puede alojar cientos de contenedores, con tiempos de arranque que pasan de minutos a milisegundos [3].

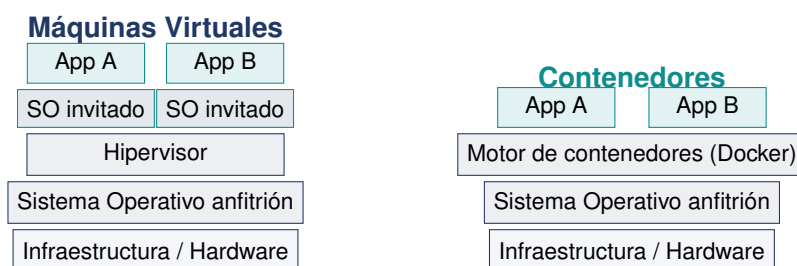


Figura 1: La VM duplica un sistema operativo invitado por instancia; el contenedor comparte el núcleo del anfitrión, eliminando esa capa pesada.

2.3. Arquitectura de Docker

Docker implementa una arquitectura **cliente-servidor**. El **cliente** (`docker` en la CLI o la integración de IntelliJ) envía órdenes; el **Docker daemon** (`dockerd`) las ejecuta: construye imágenes, crea

Tabla 1: Comparación cuantitativa orientativa entre máquinas virtuales y contenedores.

Criterio	Máquina Virtual	Contenedor
Unidad virtualizada	Hardware (SO invitado completo)	Sistema operativo (comparte núcleo)
Tamaño típico	Gigabytes	Decenas de megabytes
Arranque	Decenas de segundos a minutos	Milisegundos a segundos
Densidad por host	Decenas	Cientos
Aislamiento	Fuerte (frontera de hardware)	Bueno (namespaces/cgroups)
Caso de uso ideal	Cargas heterogéneas / SO distintos	Microservicios homogéneos y escalables

contenedores y gestiona redes y volúmenes. Las piezas que el estudiante debe distinguir con precisión son cuatro.

- **Imagen (image):** plantilla inmutable, de solo lectura, organizada en capas. Cada instrucción del Dockerfile añade una capa cacheable. Es el “molde”.
- **Contenedor (container):** instancia en ejecución de una imagen, con una capa de escritura efímera encima. Es el “objeto” creado a partir del molde.
- **Dockerfile:** receta de texto que describe cómo construir la imagen, paso a paso.
- **Registro (registry):** repositorio de imágenes (Docker Hub, GitHub Container Registry). `docker pull` descarga y `docker push` publica.

2.4. El Dockerfile y sus instrucciones esenciales

Un Dockerfile es una secuencia de instrucciones; cada una genera una capa. Conviene memorizar el núcleo: FROM (imagen base), WORKDIR (directorio de trabajo), COPY (copia archivos al contexto de la imagen), RUN (ejecuta un comando en tiempo de construcción), EXPOSE (documenta el puerto), ENV (variables de entorno), USER (usuario de ejecución) y ENTRYPOINT/CMD (proceso que arranca el contenedor). La diferencia entre ENTRYPOINT y CMD es sutil pero examinable: ENTRYPOINT define el ejecutable fijo; CMD aporta argumentos por defecto que el usuario puede sobrescribir.

Construcción multi-etapa: por qué importa

Compilar Java requiere el **JDK** (cientos de megabytes); ejecutar el `.jar` solo requiere el **JRE**. Una construcción *multi-stage* usa una etapa pesada para compilar y copia únicamente el artefacto final a una etapa de runtime ligera. Resultado: la imagen publicada es pequeña, arranca rápido y reduce la superficie de ataque (menos software = menos vulnerabilidades).

2.5. Estructura de archivos del proyecto (etapa Docker)

El microservicio `svc-estudiantes` (Spring Boot 4.0.x sobre Java 25) ya existe de la semana anterior. Para contenerizarlo se añaden dos archivos en la raíz: el Dockerfile y el `.dockerignore`.

```
svc-estudiantes/
|-- mvnw                      # wrapper de Maven (Linux/macOS)
|-- mvnw.cmd                  # wrapper de Maven (Windows)
|-- pom.xml                   # dependencias y build
|-- Dockerfile                # <-- NUEVO: receta de la imagen
|-- .dockerignore             # <-- NUEVO: que NO copiar al contexto
|-- .mvn/wrapper/             # binarios del wrapper
`-- src/
    |-- main/
        |-- java/ec/edu/uteq/svcestudiantes/
        |-- SvcEstudiantesApplication.java
        |-- controller/EstudianteController.java
```

```

|   |-- service/EstudianteService.java
|   |-- repository/EstudianteRepository.java
|   |-- model/Estudiante.java
|   `-- dto/EstudianteDTO.java
|-- resources/
|   |-- application.properties
|   `-- application-docker.properties # <-- NUEVO: perfil contenedor

```

□ svc-estudiantes/Dockerfile

```

1 # syntax=docker/dockerfile:1
2
3 # ===== Etapa 1: build (pesada, con JDK) =====
4 FROM eclipse-temurin:25-jdk AS build
5 WORKDIR /workspace
6
7 # Copiar primero el wrapper y el pom para cachear dependencias
8 COPY .mvn/ .mvn/
9 COPY mvnw pom.xml ./
10 RUN chmod +x mvnw && ./mvnw -q -B dependency:go-offline
11
12 # Copiar el código fuente y empaquetar (sin tests para acelerar la imagen)
13 COPY src ./src
14 RUN ./mvnw -q -B clean package -DskipTests
15
16 # ===== Etapa 2: runtime (ligera, solo JRE) =====
17 FROM eclipse-temurin:25-jre-alpine AS runtime
18 WORKDIR /app
19
20 # Usuario sin privilegios (buena practica de seguridad)
21 RUN addgroup -S app && adduser -S app -G app
22
23 # Copiar SOLO el artefacto final desde la etapa de build
24 COPY --from=build /workspace/target/*.jar app.jar
25
26 EXPOSE 8080
27 USER app
28 ENTRYPOINT ["java", "-jar", "/app/app.jar"]

```

□ svc-estudiantes/.dockerignore

```

target/
.git/
.idea/
*.iml
*.log
.env

```

□ svc-estudiantes/src/main/resources/application-docker.properties

```

1 # Perfil activo cuando el microservicio corre dentro de un contenedor.
2 # Los valores llegan por variables de entorno (12-factor app).
3 server.port=8080
4 spring.datasource.url=${SPRING_DATASOURCE_URL}
5 spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
6 spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
7 spring.jpa.hibernate.ddl-auto=update
8 # Endpoints de salud para sondas de Compose y Kubernetes:
9 management.endpoints.web.exposure.include=health,info,prometheus
10 management.endpoint.health.probes.enabled=true

```

2.6. Paso a paso en IntelliJ IDEA

1. **Conectar el daemon.** Instale y arranque Docker Desktop (Windows/macOS) o el Docker Engine (Linux/WSL2). En IntelliJ: *Settings* → *Build, Execution, Deployment* → *Docker* → + y seleccione *Docker for Windows / Unix socket*. Un punto verde confirma la conexión.
2. **Crear el Dockerfile.** Clic derecho sobre la raíz del proyecto → *New* → *File* → *Dockerfile*. IntelliJ reconoce el tipo y ofrece autocompletado de instrucciones.
3. **Construir la imagen.** En el margen izquierdo (gutter) del editor, junto a la línea FROM, aparece un icono de *play* doble: *Build Image...* Asigne la etiqueta `svc-estudiantes:1.0.0`.
4. **Verla y ejecutarla.** Abra la ventana *Services* (*View* → *Tool Windows* → *Services*). Bajo el nodo *Docker* verá *Images* y *Containers*. Clic derecho sobre la imagen → *Run...*, mapee el puerto 8080:8080.
5. **Inspeccionar.** En *Services*, seleccione el contenedor para ver *Logs*, *Environment*, *Files* y abrir una terminal con *Exec* → `/bin/sh`.

Comandos equivalentes en terminal

Para quienes prefieren la CLI (PowerShell por defecto; Linux/macOS como comentario):

❑ Terminal -- Windows PowerShell

```
# Construir la imagen desde la raíz del proyecto
docker build -t svc-estudiantes:1.0.0 .
# Linux/macOS: identico

# Ejecutar el contenedor publicando el puerto
docker run --rm -p 8080:8080 svc-estudiantes:1.0.0

# Probar el endpoint de salud (PowerShell usa curl.exe, no el alias)
curl.exe http://localhost:8080/actuador/health
# Linux/macOS: curl http://localhost:8080/actuador/health

# Listar imagenes y contenedores; ver consumo
docker images
docker ps
docker stats --no-stream
```

Errores típicos que penalizan en la práctica

(1) Copiar todo el proyecto antes del `pom.xml`: rompe la caché de capas y cada cambio recompila dependencias. (2) Usar la imagen JDK en runtime: imagen innecesariamente enorme. (3) Ejecutar como `root`: falla de seguridad. (4) Olvidar `.dockerignore`: se copia `target/` y `.git/`, inflando el contexto de construcción.

3. Subtema 2 — Composición de servicios: Docker Compose

3.1. Por qué Compose

Un microservicio rara vez vive solo: necesita una base de datos, quizá un gateway, una caché. Levantar cada pieza con `docker run` y conectarlas a mano es frágil y no reproducible. **Docker Compose** resuelve esto con un único archivo declarativo, `compose.yaml`, que describe *todas* los servicios, sus redes y volúmenes; un solo comando, `docker compose up`, levanta el sistema completo en el orden correcto.

Modelo declarativo

En lugar de decir *cómo* arrancar cada contenedor (imperativo), se declara el **estado deseado** del sistema (declarativo) y Compose lo materializa. Este es el mismo principio que rige Kubernetes; dominarlo aquí facilita el salto al subtema siguiente.

3.2. Anatomía de `compose.yaml`

Las claves esenciales son: `services` (cada contenedor del sistema), `image/build` (de dónde sale su imagen), `environment` (variables), `ports` (publicación al host), `expose` (visibilidad solo interna), `depends_on` (orden y, con `condition`, espera de salud), `healthcheck` (sonda de disponibilidad), `volumes` (persistencia) y `networks` (segmentación). Desde la especificación actual de Compose ya **no se escribe** la clave `version`.

3.3. Estructura de archivos del proyecto (etapa Compose)

```
svc-estudiantes/
|-- Dockerfile
|-- .dockerignore
|-- compose.yaml          # <-- NUEVO: orquestacion local del stack
|-- .env.example          # <-- NUEVO: plantilla de variables (sin secretos)
|-- infra/
|   |-- nginx/
|   |-- nginx.conf        # <-- NUEVO: configuracion del API Gateway
|-- src/ ...
```

□ `svc-estudiantes/compose.yaml`

```
1 name: svc-estudiantes-stack
2
3 services:
4   db:
5     image: postgres:17-alpine
6     environment:
7       POSTGRES_DB: ${POSTGRES_DB}
8       POSTGRES_USER: ${POSTGRES_USER}
9       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
10    volumes:
11      - db-data:/var/lib/postgresql/data
12    healthcheck:
13      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
14      interval: 10s
15      timeout: 5s
16      retries: 5
17    networks: [backend]
18
19   app:
20     build: .
21     image: svc-estudiantes:1.0.0
22     environment:
23       SPRING_PROFILES_ACTIVE: docker
24       SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/${POSTGRES_DB}
25       SPRING_DATASOURCE_USERNAME: ${POSTGRES_USER}
26       SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD}
27     depends_on:
28       db:
29         condition: service_healthy # espera a que la BD este sana
30     expose: ["8080"]               # visible solo en la red interna
31     networks: [backend]
32
```

```

33 gateway:
34   image: nginx:1.27-alpine
35   ports: ["8080:80"]           # unico puerto publicado al host
36   volumes:
37     - ./infra/nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
38   depends_on: [app]
39   networks: [backend]
40
41 volumes:
42   db-data:
43
44 networks:
45   backend:
46     driver: bridge

```

□ svc-estudiantes/infra/nginx/nginx.conf

```

1 upstream svc_estudiantes {
2   server app:8080;           # 'app' resuelve por DNS interno de Compose
3 }
4
5 server {
6   listen 80;
7
8   location /api/ {
9     proxy_pass      http://svc_estudiantes;
10    proxy_set_header Host $host;
11    proxy_set_header X-Real-IP $remote_addr;
12  }
13
14  location /actuator/health {
15    proxy_pass http://svc_estudiantes/actuator/health;
16  }
17 }

```

□ svc-estudiantes/.env.example

```

# Copie este archivo a .env y rellene valores reales.
# .env NUNCA debe subirse al repositorio (esta en .gitignore).
POSTGRES_DB=estudiantes
POSTGRES_USER=uteq
POSTGRES_PASSWORD=cambie-esta-clave

```

Red interna y resolución por nombre

Compose crea una red bridge donde cada servicio es alcanzable por su **nombre**: la aplicación se conecta a la base de datos usando el host db, no localhost ni una IP. Esa resolución por DNS interno es la base del descubrimiento de servicios y se generaliza después en Kubernetes.

3.4. Paso a paso en IntelliJ IDEA

1. **Abrir el archivo.** Al abrir `compose.yaml`, IntelliJ lo reconoce y muestra iconos de *play* en el gutter, tanto a nivel del archivo (todo el stack) como por servicio.
2. **Levantar el stack.** Clic en el *play* de la primera línea → *Run*. IntelliJ construye la imagen `app` (porque tiene `build: .`), descarga `postgres` y `nginx`, y arranca en orden respetando `depends_on`.
3. **Observar.** En *Services*, bajo el nodo del proyecto Compose, verá los tres contenedores con su estado de salud. Seleccione cada uno para *Logs*, *Dashboard* y *terminal*.

4. **Probar.** Use el cliente HTTP de IntelliJ (archivo `.http`) o el navegador contra `http://localhost:8080/api/estudiantes`: la petición entra por Nginx y se enruta al microservicio.
5. **Detener y limpiar.** El botón *stop* en *Services* ejecuta `compose down`; añada la opción de eliminar volúmenes solo si desea borrar los datos.

❏ Terminal -- Windows PowerShell

```
# Construir y levantar el stack completo en segundo plano
docker compose up --build -d

# Ver estado y salud de los servicios
docker compose ps

# Seguir los logs del microservicio
docker compose logs -f app

# Escalar el microservicio a 3 replicas (gateway balancea entre ellas)
docker compose up -d --scale app=3

# Derribar el stack (conserva el volumen de datos)
docker compose down
```

Del laboratorio a producción

Compose es excelente para desarrollo y para un solo host, pero **no** reprograma contenedores si una máquina cae, ni reparte la carga entre varios servidores, ni hace despliegues progresivos. Cuando el sistema debe ser *altamente disponible* y multi-nodo, se necesita un orquestador: Kubernetes.

4. Subtema 3 — Introducción a Kubernetes: pods, deployments y servicios

4.1. Qué añade Kubernetes

Kubernetes (abreviado **K8s**) es un orquestador de contenedores que mantiene automáticamente el *estado deseado* del sistema sobre un clúster de máquinas. Si un contenedor muere, lo reprograma; si una máquina cae, mueve sus cargas a otra; si la demanda sube, escala réplicas. Nació de la experiencia de Google operando contenedores a gran escala con sus sistemas internos Borg y Omega [4, 2].

4.2. Arquitectura del clúster

Un clúster tiene dos planos. El **plano de control** decide *qué* debe ocurrir; los **nodos de trabajo** ejecutan las cargas. El estudiante debe ubicar cada componente.

Plano de control: kube-apiserver (puerta de entrada única; toda orden pasa por aquí), etcd (almacén clave-valor que guarda el estado del clúster), scheduler (decide en qué nodo colocar cada Pod) y controller-manager (bucles que reconcilian estado real con estado deseado). **Nodo de trabajo:** kubelet (agente que arranca y vigila los Pods del nodo), kube-proxy (reglas de red para los Services) y un *runtime* de contenedores (por ejemplo containerd).

4.3. Objetos fundamentales

Pod

la unidad mínima de despliegue: uno o varios contenedores que comparten red y almacenamiento. Los Pods son *efímeros*; no se gestionan directamente en producción.

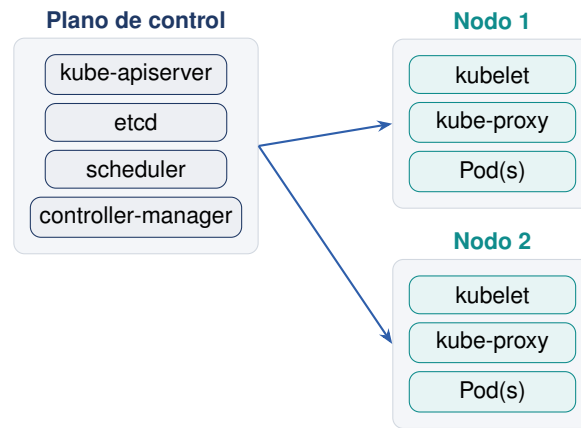


Figura 2: Plano de control (decide) frente a nodos de trabajo (ejecutan). El apiserver es la única puerta de entrada al clúster.

ReplicaSet

garantiza que un número dado de réplicas idénticas de un Pod esté siempre en ejecución.

Deployment

gestiona ReplicaSets y añade *actualizaciones controladas* (rolling update) y *rollback*. Es el objeto que el estudiante usará para sus microservicios.

Service

da una dirección de red estable y balanceo a un conjunto cambiante de Pods, seleccionados por etiquetas. Tipos: ClusterIP (interno), NodePort (expone un puerto en cada nodo) y LoadBalancer (IP externa en la nube).

ConfigMap / Secret

externalizan configuración y credenciales fuera de la imagen.

Ingress

enruta tráfico HTTP externo hacia Services por host y ruta (el equivalente al gateway).

4.4. Estructura de archivos del proyecto (etapa Kubernetes)

```
svc-estudiantes/
|-- k8s/
|   |-- namespace.yaml      # aislamiento logico del proyecto
|   |-- configmap.yaml      # configuracion no sensible
|   |-- secret.yaml         # credenciales
|   |-- deployment.yaml     # microservicio: 3 replicas + sondas
|   |-- service.yaml        # IP estable + balanceo (ClusterIP)
|   |-- ingress.yaml        # entrada HTTP externa (opcional)
```

❏ svc-estudiantes/k8s/deployment.yaml

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: svc-estudiantes
5   namespace: distapp
6   labels:
7     app: svc-estudiantes
8 spec:
9   replicas: 3
10  selector:
11    matchLabels:
12      app: svc-estudiantes
```

```

13 template:
14   metadata:
15     labels:
16       app: svc-estudiantes
17   spec:
18     containers:
19       - name: svc-estudiantes
20         image: svc-estudiantes:1.0.0
21         ports:
22           - containerPort: 8080
23         envFrom:
24           - configMapRef:
25               name: svc-estudiantes-config
26           - secretRef:
27               name: svc-estudiantes-secret
28         readinessProbe:
29           httpGet:
30             path: /actuator/health/readiness
31             port: 8080
32             initialDelaySeconds: 20
33             periodSeconds: 10
34         livenessProbe:
35           httpGet:
36             path: /actuator/health/liveness
37             port: 8080
38             initialDelaySeconds: 30
39             periodSeconds: 15
40         resources:
41           requests:
42             cpu: "250m"
43             memory: "384Mi"
44           limits:
45             cpu: "1"
46             memory: "768Mi"

```

□ svc-estudiantes/k8s/service.yaml

```

1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: svc-estudiantes
5   namespace: distapp
6 spec:
7   type: ClusterIP
8   selector:
9     app: svc-estudiantes      # enruta a todos los Pods con esta etiqueta
10  ports:
11    - port: 80                # puerto del Service
12      targetPort: 8080        # puerto del contenedor

```

□ svc-estudiantes/k8s/configmap.yaml

```

1 apiVersion: v1
2 kind: ConfigMap
3 metadata:
4   name: svc-estudiantes-config
5   namespace: distapp
6 data:
7   SPRING_PROFILES_ACTIVE: "docker"
8   SPRING_DATASOURCE_URL: "jdbc:postgresql://postgres:5432/estudiantes"

```

El contrato etiqueta-selector

El Service no conoce IPs de Pods: declara un selector por etiqueta (`app: svc-estudiantes`) y K8s mantiene al día el conjunto de destinos. Este *acoplamiento por etiquetas*, no por dirección, es lo que permite escalar, reemplazar y desplegar Pods sin tocar la red. También es la palanca de los patrones blue-green y canary del subtema 4.

4.5. Paso a paso en IntelliJ IDEA

1. **Clúster local.** Para practicar sin nube, instale minikube o kind y arránquelo (`minikube start`). Esto genera un kubeconfig en `~/.kube/config`.
2. **Plugin Kubernetes.** En IntelliJ IDEA Ultimate viene incluido; en otras ediciones, instálelo desde *Settings* → *Plugins*. Reconoce automáticamente el kubeconfig.
3. **Aplicar manifiestos.** Abra cualquier `*.yaml` de `k8s/`: en el gutter aparece un icono para *Apply* el recurso al clúster. También puede arrastrar la carpeta completa.
4. **Explorar.** En *Services* → *Kubernetes* verá *Workloads* (Deployments, Pods), *Network* (Services, Ingress) y *Config* (ConfigMaps, Secrets). Doble clic en un Pod abre sus logs; *Port Forward* expone el Service en localhost para probarlo.
5. **Editar en vivo.** El editor valida el esquema y autocompleta campos (`apiVersion`, `kind`), reduciendo errores de indentación YAML.

□ Terminal -- Windows PowerShell

```
# Cargar la imagen local en minikube (sin registro remoto)
minikube image load svc-estudiantes:1.0.0

# Aplicar todos los manifiestos del directorio
kubectl apply -f k8s/

# Ver el estado de los objetos en el namespace
kubectl get pods,deploy,svc -n distapp

# Seguir el despliegue (rolling update) y leer logs
kubectl rollout status deployment/svc-estudiantes -n distapp
kubectl logs -f deploy/svc-estudiantes -n distapp

# Exponer el Service en localhost para probarlo
kubectl port-forward svc/svc-estudiantes 8080:80 -n distapp
```

5. Subtema 4 — Patrones de despliegue distribuido (blue-green y canary)

5.1. El problema: liberar sin tiempo de caída

Actualizar un sistema distribuido en producción plantea un dilema: detenerlo para cambiar la versión viola el **SLA** de disponibilidad, pero cambiarlo en caliente arriesga propagar un fallo a todos los usuarios. Los *patrones de despliegue* resuelven el dilema controlando *qué fracción* del tráfico ve la versión nueva y *con qué rapidez* se puede revertir [7, 8].

5.2. Estrategia base de Kubernetes: rolling update

Por defecto, un Deployment actualiza con **rolling update**: reemplaza los Pods viejos por nuevos de forma gradual, manteniendo siempre un mínimo disponible (`maxUnavailable`) y un máximo de Pods extra (`maxSurge`). Es seguro y automático, pero mezcla ambas versiones durante la transición y no permite “probar con pocos usuarios antes de comprometerse”. Para eso existen blue-green y canary.

5.3. Despliegue Blue-Green

Blue-Green

Se mantienen **dos entornos idénticos**: *blue* (versión actual, recibe todo el tráfico) y *green* (versión nueva, desplegada en paralelo y verificada sin tráfico real). Cuando *green* pasa las pruebas, se **conmuta el tráfico** de golpe (por ejemplo cambiando el selector del Service). Si algo falla, el *rollback* es instantáneo: se vuelve a apuntar a *blue*.

En Kubernetes se implementa con dos Deployments que comparten la etiqueta app pero difieren en version; el Service selecciona una sola versión a la vez.

❏ svc-estudiantes/k8s/blue-green/service.yaml

```

1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: svc-estudiantes
5   namespace: distapp
6 spec:
7   type: ClusterIP
8   selector:
9     app: svc-estudiantes
10    version: blue           # <-- la palanca: cambiar a 'green' conmuta todo
11   ports:
12     - port: 80
13       targetPort: 8080

```

❏ Terminal -- conmutación y rollback

```

# 1) Desplegar green (version 1.1.0) en paralelo, sin trafico todavia
kubectl apply -f k8s/blue-green/deployment-green.yaml

# 2) Verificar green internamente (port-forward al Pod de green) ...

# 3) Conmutar TODO el trafico a green cambiando el selector del Service
kubectl patch service svc-estudiantes -n distapp `
  -p '{"spec":{"selector":{"app":"svc-estudiantes","version":"green"}}}'
# Linux/macOS: misma orden sin el backtick de continuacion

# 4) Rollback instantaneo si algo falla: volver a blue
kubectl patch service svc-estudiantes -n distapp `
  -p '{"spec":{"selector":{"app":"svc-estudiantes","version":"blue"}}}'

```

Ventajas: conmutación y rollback instantáneos; la versión nueva se valida en un entorno idéntico antes de recibir usuarios. **Desventajas:** duplica los recursos durante la transición; un cambio de esquema de base de datos exige que ambas versiones sean compatibles con el mismo esquema.

5.4. Despliegue Canary

Canary

La versión nueva (*canary*) recibe un **pequeño porcentaje** del tráfico real (por ejemplo 10 %) mientras la *stable* atiende el resto. Se observan métricas (errores, latencia); si el canary se comporta bien, se incrementa su proporción de forma progresiva hasta el 100 %; si no, se retira afectando solo a una minoría de usuarios. El nombre alude al “canario en la mina”.

La forma más simple en K8s vanilla es jugar con el *número de réplicas*: un Service que selecciona la etiqueta común app reparte tráfico proporcionalmente entre Pods stable y canary. Para control fino

de porcentajes independiente del conteo de Pods se usan pesos de Ingress (anotaciones de canary) o herramientas como *Argo Rollouts*.

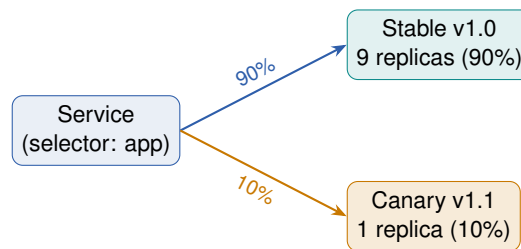


Figura 3: Canary por réplicas: el Service reparte el tráfico en proporción al número de Pods de cada versión que comparten la etiqueta app.

❏ svc-estudiantes/k8s/canary/deployment-canary.yaml

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: svc-estudiantes-canary
5    namespace: distapp
6    labels:
7      app: svc-estudiantes      # MISMA etiqueta 'app' que stable -> el Service lo incluye
8      track: canary
9  spec:
10     replicas: 1                # 1 de 10 Pods totales ~= 10% del trafico
11     selector:
12       matchLabels:
13         app: svc-estudiantes
14         track: canary
15     template:
16       metadata:
17         labels:
18           app: svc-estudiantes
19           track: canary
20       spec:
21         containers:
22           - name: svc-estudiantes
23             image: svc-estudiantes:1.1.0
24             ports:
25               - containerPort: 8080
  
```

Tabla 2: Comparación de estrategias de despliegue distribuido.

Criterio	Rolling update	Blue-Green	Canary
Tiempo de caída	Cero	Cero	Cero
Coste de recursos	Bajo	Alto (doble)	Medio
Velocidad de rollback	Media	Instantánea	Alta
Exposición al riesgo	Gradual	Total al conmutar	Mínima (pocos usuarios)
Prueba con tráfico real	No selectiva	No (antes de conmutar)	Sí, controlada
Complejidad	Baja (nativa)	Media	Alta (métricas)

Regla de oro para ambos patrones

Toda liberación progresiva exige **compatibilidad hacia atrás**: mientras dos versiones conviven, deben tolerar el mismo esquema de datos y el mismo contrato de API. Los cambios de esquema se hacen en pasos *aditivos* (expand) antes de eliminar lo viejo (contract).

6. Cierre, actividad y evaluación

6.1. Actividad de clase sugerida

Sobre el microservicio `svc-estudiantes`, el estudiante debe: (1) escribir el `Dockerfile` multi-etapa y construir la imagen desde IntelliJ; (2) levantar el stack con `compose.yaml` (microservicio + PostgreSQL + Nginx) y probar un flujo POST/GET vía gateway; (3) desplegar el Deployment y el Service en minikube y comprobar 3 réplicas sanas; y (4) ejecutar una conmutación blue-green y un canary de 1 réplica, documentando con capturas el estado antes y después. Entregable: repositorio con `Dockerfile`, `compose.yaml`, carpeta `k8s/` y un `README.md` reproducible.

6.2. Preguntas de comprobación y defensa

1. **Conceptual:** ¿Por qué un contenedor arranca en milisegundos y una VM en decenas de segundos? Relacione la respuesta con la Figura 1.
2. **Diseño:** En el `Dockerfile`, ¿qué gana exactamente la etapa de runtime al usar `eclipse-temurin:25-jre-alpine` en vez de `25-jdk`? Cuantifíquelo.
3. **Compose:** ¿Por qué la aplicación se conecta a la base de datos usando el `host db` y no `localhost`? ¿Qué mecanismo lo hace posible?
4. **Kubernetes:** Si un Pod del Deployment muere, ¿qué componente lo detecta y qué objeto garantiza que vuelva a haber 3 réplicas?
5. **Service:** Explique por qué el Service no necesita conocer las IPs de los Pods. ¿Qué papel cumple el selector?
6. **Patrones:** Un equipo libera v1.1 y descubre un fallo grave 30 segundos después. ¿Qué estrategia le da el rollback más rápido: blue-green o canary? Justifique.
7. **Crítica:** ¿Por qué ambos patrones exigen compatibilidad de esquema de base de datos entre versiones? Dé un ejemplo de cambio que rompa la convivencia.

Referencias

- [1] D. Merkel, "Docker: lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, art. 2, Mar. 2014.
- [2] D. Bernstein, "Containers and cloud: From LXC to Docker to Kubernetes," *IEEE Cloud Computing*, vol. 1, no. 3, pp. 81–84, Sep. 2014, doi: 10.1109/MCC.2014.51.
- [3] C. Pahl, A. Brogi, J. Soldani, and P. Jamshidi, "Cloud container technologies: A state-of-the-art review," *IEEE Transactions on Cloud Computing*, vol. 7, no. 3, pp. 677–692, Jul.–Sep. 2019, doi: 10.1109/TCC.2017.2702586.
- [4] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, May 2016, doi: 10.1145/2890784.
- [5] B. Burns, J. Beda, K. Hightower, and L. Evenson, *Kubernetes: Up and Running*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022. ISBN: 978-1-098-11020-8.
- [6] B. Burns, *Designing Distributed Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2025. ISBN: 978-1-09-815635-0.
- [7] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018. ISBN: 978-1-617-29454-1.
- [8] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021. ISBN: 978-1-4920-3402-5.

[9] G. C. Guerrero Ulloa, *Aplicaciones Distribuidas: Notas de Curso*, Cap. 7. Quevedo, Ecuador: UTEQ, 2026.